# Tremor, Frequency & Machine Learning

How our wrist band measures a Parkinson's tremor — and how a neural network would do the same job. Explained from scratch, no physics needed.

ENGF0031 Scenario 2 • Cadence wrist monitor • A briefing for the team.

**How to use this.** Read it top to bottom and you can explain the whole pipeline to anyone. Each section opens with the idea in one breath, then a picture. Example numbers are marked *illustrative* — the *real* sensitivity/specificity come from our own wrist recording and go in the worksheet, not here.

## 1 What we are catching: a Parkinson's tremor

Many people with Parkinson's disease have a **resting tremor**: the hand shakes by itself, rhythmically, *while it is resting* in the lap. It is fast and small — the hand trembles roughly **4 to 6 times every second** — and, oddly, it usually *calms down* the moment the person reaches out to do something on purpose. Clinicians track how strong and how frequent this tremor is to see whether medication is working.

Our device, **Cadence**, is a small computer worn **on the wrist** (think of a smartwatch). It watches the hand, measures how strong the tremor is, and *reports* it to a dashboard. That is all — it does not buzz or shock or correct anything. Measuring-and-reporting with no action back to the body is called an **open loop** (a fitness tracker is open-loop; a thermostat that switches the heater is *closed*-loop).

> **In plain English:** It is a thermometer for a tremor. A thermometer doesn't *change* your temperature, it just reports it honestly so a doctor can decide. Cadence reports how much the hand is shaking at the tremor speed.

The single most important fact for the whole project:

> **A Parkinson's tremor has a *fingerprint*: it is rhythmic, and it happens at one particular speed (about 4–6 wiggles per second).** Random fidgeting or reaching for a cup does not. So if we can measure "how much shaking is happening *at the tremor speed*", we can tell tremor apart from ordinary life. Everything below is how we measure that one thing.

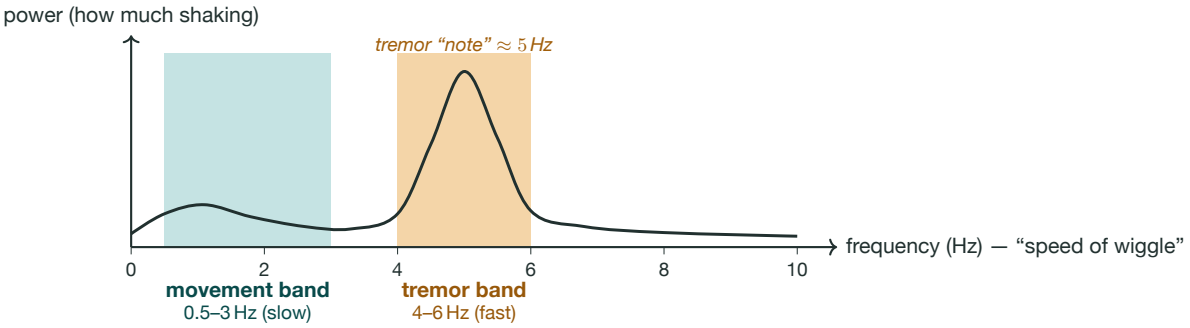
## 2 Frequency: shaking has a "speed"

**Frequency** just means *how many wiggles happen per second*. We measure it in **hertz (Hz)**: 5 Hz means back-and-forth five times a second. Slow wiggles are a low frequency; fast wiggles are a high frequency.

**In plain English:** Think of sound. A deep bass note is a slow vibration (low frequency); a high treble note is a fast vibration (high frequency). A tremor is like a single, steady note your wrist keeps humming at about 5 Hz.

Here is the trick that makes the whole field possible. **Any messy, wiggly signal can be seen as a stack of pure, simple wiggles of different speeds, all added together.** Pull them back apart and you can ask "how much of each speed is in here?" That "how much of each speed" picture is called a **spectrum** (or *power spectrum*).

**In plain English:** A spectrum is the bar-graph equaliser on a music app. Music is many notes mixed together; the equaliser shows how loud each pitch is. A spectrum does the same for your wrist: it shows how strong the shaking is at each speed.




(Illustrative spectrum of a resting hand with a tremor: a sharp bump sits in the 4–6 Hz band. Voluntary movement instead puts its energy down in the slow 0.5–3 Hz band.)

## 3 Energy in a band: turning shaking into ONE number

We don't need the whole spectrum — we only care about *one* part of it: the **4–6 Hz tremor band**. We add up how strong the shaking is across just that band and call it the **tremor power**: one number that means "how much tremor-speed shaking is going on right now". Big number = strong tremor; near-zero = a calm hand.

How does a tiny battery chip compute that, live, without a maths supercomputer? Two cheap steps:

1. **A band-pass filter — a gatekeeper.** A filter is a piece of maths that lets some wiggle-speeds through and blocks the rest. Ours is a *band-pass* filter tuned to 4–6 Hz: it throws away slow drift and arm-swinging (too slow) *and* fast electrical noise (too fast), and keeps only tremor-speed wiggle. **In plain English:** It's like noise-cancelling headphones set to keep *only* a 5 Hz hum and mute everything else.
2. **Square and sum — measure the leftover.** Whatever survives the filter, we *square* each sample (squaring makes every value positive and makes big swings count much more than small ones — that is what "energy" means) and add them all up. The total is the tremor power.

$$\text{tremor power} = \sum_{\text{samples in the window}} \left(\text{wiggle left after the 4–6 Hz filter}\right)^2$$

We compute this over a **4-second window** of data and refresh it every **2 seconds**, so the dashboard updates twice as often as the window is long. (Four seconds is long enough to *see* a 5 Hz rhythm clearly; you can't judge a rhythm from a single instant.)

> **In plain English:** A band-pass filter + "square and add up" is the cheap way to ask "how loud is the 4–6 Hz band?" without computing the whole equaliser. It's a few multiplications per sample — easy for a coin-cell chip, and exactly what our firmware (`cpx_tremor_monitor.ino`) does.

### Two small honesty checks before we believe it

- **Is the hand actually at rest? (the rest gate.)** A *resting* tremor only counts when the hand is resting. If the person is busy waving or walking, there's lots of slow 0.5–3 Hz movement — so we check that band too, and if it's high we say "moving, ignore the tremor reading" rather than cry wolf.
- **Is it sustained? (debounce.)** One odd 2-second window shouldn't flip the verdict. We require **2 windows in a row** over the line to declare "tremor present", and 2 clear ones to declare it gone. That stops the readout flickering.

### Where the score comes from

To grade the device we set a **threshold** (a line) on the tremor power and, window by window, compare what the device says to the truth. That gives the two headline numbers the worksheet asks for: **sensitivity** ("of the real tremor windows, how many did we catch?") and **specificity** ("of the calm/normal windows, how many did we correctly leave alone?"). We report those two, *not* "accuracy", because tremor windows are rare — a lazy detector that says "never" would score high accuracy while catching nothing.

## 4 Machine learning: a different way to make the rule

Everything above is a **hand-written rule**: *we*, the humans, studied the problem and decided "filter to 4–6 Hz, measure the power, compare to a line". There is a completely different philosophy, and it is what people mean by **machine learning (ML)**.

**Classic way (what Cadence ships)**

A human writes the rule from physics: "if 4–6 Hz power $>$ line $\Rightarrow$ tremor". You can read it and know exactly why it fired.

**Machine-learning way**

Nobody writes the rule. You *show* the computer thousands of labelled clips ("this $=$ tremor", "this $=$ not") and it *tunes itself* until it predicts the labels.

**In plain English:** Teaching a kid the word "dog" the classic way means listing rules: four legs, fur, a tail… (and a cat sneaks through). The machine-learning way is to show them hundreds of photos labelled "dog / not dog" until they just *get* it. ML learns from examples, not from a rulebook.

How does it "tune itself"? Inside the model are thousands of little numbers called **weights** — think tiny knobs. **Training** is a loop: the model guesses, we compare its guess to the true label, and we nudge every knob a hair in the direction that would have made the guess better. Do that over millions of examples and the knobs settle into a setting that works.

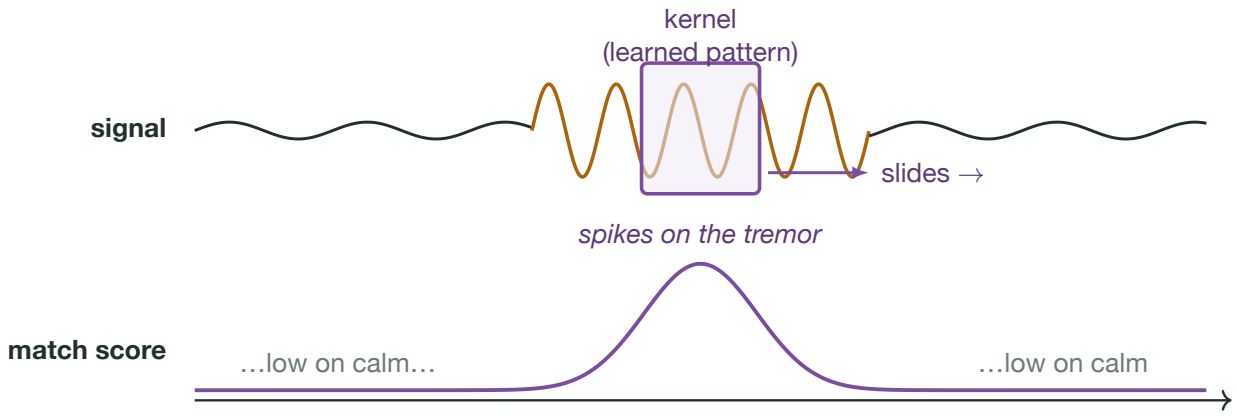
> **The trade.** ML can learn patterns a human would never think to write down — but it needs *a lot* of labelled examples to learn from, more memory and computing power to run, and it can't easily *tell* you why it decided something. A hand-written rule is the opposite: limited, but tiny, instant, and fully explainable.

## 5 CNN: a pattern-finder for signals

A **neural network** is just a big stack of those weighted knobs arranged in layers, where each layer's output feeds the next. A **convolutional neural network (CNN)** is a *special* kind that is brilliant at finding **local patterns** in grid-shaped data — pixels in an image (a 2-D grid) or, for us, **samples over time** (a 1-D grid). For accelerometer signals we'd use a *1-D* CNN.

The key move is the **convolution**: take a small "pattern detector" (called a **kernel** or **filter**) and *slide it along* the signal, scoring how well the signal matches that pattern at every position. Where the pattern appears, the score spikes.

**In plain English:** A convolution is a little transparent stencil you slide across the signal. Wherever the shape underneath matches the stencil, it lights up. Slide a "5 Hz burst" stencil along a wrist trace and it lights up exactly on the tremor.




(A convolution slides a small learned pattern across the signal and reports how well it matches at each point. A real CNN stacks many such detectors.)

A CNN stacks these: the first layer's kernels find tiny features (a short up-down wiggle); the next layer combines those into bigger ideas ("a sustained 5 Hz burst lasting two seconds"); a final layer turns that into a verdict, "tremor / not". Crucially, **it learns the kernels itself** from labelled data — nobody hand-draws the stencils.

**The punchline that connects everything.** Our hand-built 4–6 Hz band-pass filter *is* a convolution — one stencil, designed by us from physics. A 1-D CNN would *learn a whole stack* of stencils from data. Fed enough labelled wrist recordings, it might rediscover "watch the 4–6 Hz band" on its own *and* pick up subtler cues (how the tremor ramps up, how it differs from a shiver) that a single fixed filter misses. Same family of maths — one hand-tuned, one learned.

## 6 So which does Cadence use — and why?

We ship the **hand-built filter**, not a CNN. Here is the honest side-by-side so the team can defend the choice:

| | **Hand-built filter** (what we ship) | **1-D CNN** (the ML alternative) |
|---|---|---|
| How the rule is made | A human, from physics | Learned from labelled data |
| Needs training data? | No | Yes — and lots of it |
| Runs on the coin-cell CPX? | Yes, a few sums per sample | Heavy: needs more memory & compute |
| Can it explain itself? | Fully ("4–6 Hz power > line") | Mostly a black box |
| Best when… | you understand the signal, have little data, and must justify every reading | you have lots of labelled data and want to catch subtle patterns |

For a **one-week clinical prototype** the filter wins on every axis that matters to us: we have very little labelled tremor data, the code must run on the tiny Circuit Playground, and a clinician has to *trust* and justify every number it reports — which is easy when the rule is "power in the tremor band crossed a line". The CNN is the honest *"given more data and time"* upgrade, not a corner we cut.

**In plain English:** We didn't avoid machine learning because it's hard — we chose the simpler tool because it's the *right* one for a tiny, explainable, low-data medical monitor. Knowing *when not* to reach for the fancy method is itself good engineering.

## 7 One-page cheat sheet

| | |
|---|---|
| **Resting tremor** | the hand shaking by itself at rest, about 4–6 times a second; calms during deliberate movement. This is what we measure. |
| **Frequency (Hz)** | how many wiggles per second. Movement = slow (0.5–3 Hz); tremor = fast (4–6 Hz). |
| **Spectrum** | the "equaliser" picture: how much shaking sits at each frequency. |
| **Band-pass filter** | a gatekeeper that keeps only 4–6 Hz wiggle and blocks the rest. |
| **Tremor power** | filter to 4–6 Hz, square each sample, add them up. One number for "how much tremor". |
| **Window** | 4 s of data, refreshed every 2 s, so the readout updates often with enough context to judge a rhythm. |
| **Rest gate** | ignore the tremor reading while the hand is actively moving (lots of slow-band power). |
| **Debounce** | need 2 windows in a row to change the verdict; stops flicker. |
| **Sensitivity** | catch rate: of real tremor windows, how many we caught. |
| **Specificity** | leave-alone rate: of calm windows, how many we correctly ignored. (We report these two, not "accuracy".) |
| **Machine learning** | don't hand-write the rule — learn it from thousands of labelled examples by nudging internal "knobs" (weights). |
| **CNN** | a network that finds patterns by sliding small learned "stencils" (kernels) across the data. Our fixed filter is one stencil; a CNN learns a stack of them. |

Built for ENGF0031 Scenario 2 "Smart Clothing". This is a teammate explainer; the spectrum and CNN sketches are illustrative. Real device performance (sensitivity / specificity) comes from our own wrist recording and is reported in the accuracy worksheet, not here. The ankle / freeze-of-gait version of this theory lives in `cadence_theory.tex`.